Lab Sheet-07

Data Structure I (BTCS209P)

B.Tech III Semester (Session 2026-2027)

Objective:

To understand and implement fundamental Tree data structures and operations using C++, and to analyze their working, algorithms, traversal techniques, time complexity, and space complexity.

Requirements:

C++ Compiler / IDE
VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment
Basic knowledge of arrays, loops, functions, pointers, structures/classes, and recursion

Concepts Covered:

1. Tree and Terminology
2. Binary Tree
3. Binary Search Tree (BST)
4. Tree Traversals
5. Insertion and Searching in BST
6. Deletion in BST
7. Height and Node Counting

1. Tree

A Tree is a non-linear data structure consisting of nodes connected by edges. It represents hierarchical relationships between data elements.

Important terms include root, parent, child, leaf, sibling, degree, level, height, and subtree.

2. Binary Tree

A Binary Tree is a tree in which each node has at most two children, called the left child and right child.

Node Structure:

```
struct Node
{
   int data;
   Node* left;
   Node* right;
};
```

3. Binary Search Tree (BST)

1. A Binary Search Tree is a binary tree in which:
   • All elements in the left subtree are smaller than the root.
   • All elements in the right subtree are greater than the root.
   • The same property is recursively followed by every subtree.

Example:

```
        50
       /  \
     30    70
    / \    / \
  20  40  60  80
```

4. Tree Traversals

Tree traversal means visiting every node of a tree in a systematic order.

4.1 Inorder Traversal
Order: Left → Root → Right
For a BST, inorder traversal produces elements in sorted order.

Algorithm:
INORDER(root)
1. If root is not NULL
2. INORDER(root->left)
3. Visit root
4. INORDER(root->right)
5. Stop

4.2 Preorder Traversal
Order: Root → Left → Right

Algorithm:
PREORDER(root)
1. If root is not NULL
2. Visit root
3. PREORDER(root->left)
4. PREORDER(root->right)
5. Stop

4.3 Postorder Traversal
Order: Left → Right → Root

Algorithm:
POSTORDER(root)
1. If root is not NULL
2. POSTORDER(root->left)
3. POSTORDER(root->right)

4. Visit root
5. Stop

5. Insertion in Binary Search Tree

A new node is inserted according to the BST property.

Algorithm:
INSERT(root, key)
1. If root is NULL
2. Create a new node
3. Return new node
4. If key < root->data
5. root->left = INSERT(root->left, key)
6. Else if key > root->data
7. root->right = INSERT(root->right, key)
8. Return root

Time Complexity:
• Best/Average: O(log n)
• Worst: O(n)

6. Searching in Binary Search Tree

Searching compares the required value with the current node and moves either to the left or right subtree.

Algorithm:
SEARCH(root, key)
1. If root is NULL
2. Return false
3. If root->data == key
4. Return true
5. If key < root->data
6. Search in left subtree
7. Else
8. Search in right subtree

Time Complexity:
• Best/Average: O(log n)
• Worst: O(n)

7. Deletion in Binary Search Tree

Deletion from a BST has three main cases:
1. Node is a leaf node
2. Node has one child
3. Node has two children

For two children, the node can be replaced by its inorder successor or inorder predecessor.

Time Complexity:
• Best/Average: O(log n)
• Worst: O(n)

8. Height of a Binary Tree

The height of a tree represents the longest path from the root to a leaf.

Algorithm:
HEIGHT(root)
1. If root is NULL
2. Return 0
3. leftHeight = HEIGHT(root->left)
4. rightHeight = HEIGHT(root->right)
5. Return 1 + MAX(leftHeight, rightHeight)

Time Complexity: O(n)
Space Complexity: O(h) due to recursion stack, where h is the height of the tree.

Lab Exercises

8. Write a C++ program to create a Binary Tree using dynamically allocated nodes and display its structure.
9. Write a C++ program to implement Inorder Traversal of a Binary Tree.
10. Write a C++ program to implement Preorder Traversal of a Binary Tree.
11. Write a C++ program to implement Postorder Traversal of a Binary Tree.
12. Write a C++ program to implement all three traversals — Inorder, Preorder, and Postorder — for the same Binary Tree.
13. Write a C++ program to create a Binary Search Tree (BST) by inserting n elements.
14. Write a C++ program to perform searching in a Binary Search Tree for a given key.
15. Write a C++ program to perform insertion in a Binary Search Tree and display its inorder traversal.
16. Write a C++ program to delete a leaf node from a Binary Search Tree.
17. Write a C++ program to delete a node having one child from a Binary Search Tree.
18. Write a C++ program to delete a node having two children from a Binary Search Tree using the inorder successor.
19. Write a C++ program to calculate the height of a Binary Tree.
20. Write a C++ program to count the total number of nodes in a Binary Tree.
21. Write a C++ program to count the number of leaf nodes in a Binary Tree.
22. Write a C++ program to implement a menu-driven Binary Search Tree with the following operations: Insert, Search, Inorder Traversal, Preorder Traversal, Postorder Traversal, Delete, Height, Count Nodes, Exit.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing:

- Aim / Objective
- Algorithm
- C++ Program
- Sample Input
- Sample Output